

DATA247 App

Security Scan Report

Written in plain English — no tech experience needed

App Name	DATA247
Package	app.data247.prep
Date	June 8, 2026
Security Score	52 / 100
Tool Used	MobSF v4.5.0 (Mobile Security Framework)

1. What Is This Report?

This report was created after running an automated security scan on the DATA247.apk file. The scan checked the app for common security problems that hackers could exploit.

Think of it like a health checkup for your app. The tool we used (called MobSF) looks inside the app code and checks for anything risky — even without running the app on a real phone.

This report explains each problem in simple words and tells you exactly what to change in your code to fix it. You do NOT need to be a security expert to follow these steps.

2. Overall Score: 52/100 — Needs Improvement

Your app scored 52 out of 100. That means there are real security issues that should be fixed before more people use the app. The good news is: none of these are extremely severe, and all of them can be fixed with small code changes.

 **HIGH**
1 issue

 **WARNING**
5 issues

 **INFO**
2 issues

<i>Fix immediately</i>	<i>Fix soon</i>	<i>Good to fix</i>
------------------------	-----------------	--------------------

3. Issues Found — Plain English Explanations & Fixes

Each issue below has: what it means, why it matters, and exactly what to change in your code.

Issue #1 HIGH — App Supports Very Old Android Versions

What this means in simple words:

Your app is set up to work on very old Android phones (Android 6, released in 2015). Those old phones have hundreds of security holes that Google no longer fixes. If someone uses your app on one of those phones, attackers can exploit those holes to steal data.

Where to fix it — open your app's build.gradle file and find this line:

```
minSdkVersion 23
```

Change it to:

```
minSdkVersion 29
```

That's it! This tells Android that your app only works on Android 10 and newer, which are much safer phones.

 **Issue: App supports Android 6.0 (minSdk=23)**

HIGH

 Fix: Change minSdkVersion to 29 in build.gradle

```
minSdkVersion 29    // was 23 - change to this
```

Issue #2 WARNING — App Signing Uses Old Method (Janus Vulnerability)

What this means in simple words:

When you published your app, you 'signed' it — like putting a seal on a letter to prove it came from you. But your app uses an old signing method (called v1) that has a known flaw. A hacker could take your app, modify it to add malicious code, and re-upload it — and the signature would still look valid. This is called the Janus vulnerability.

How to fix it — when you build your app in Android Studio, make sure you use v2 or v3 signing only. In your build.gradle file inside the android block, add or update this:

```
android {
```

```

signingConfigs {
    release {
        v1SigningEnabled false    // turn OFF old method
        v2SigningEnabled true     // turn ON new method
        v3SigningEnabled true     // even better
    }
}
}

```

Issue: v1 signature scheme (Janus vulnerability)

WARNING

✅ Fix: Disable v1 signing, enable v2/v3 in build.gradle signingConfigs

v1SigningEnabled false | v2SigningEnabled true | v3SigningEnabled true

Issue #3 **WARNING — Anyone Can Back Up Your App's Data**

What this means in simple words:

Right now, if someone connects their phone to a computer and runs a simple command, they can copy ALL the data your app stores — login info, saved answers, quiz progress, everything. This is because a setting called allowBackup is turned on by default.

How to fix it — open your AndroidManifest.xml file. Find the line that has application and add this:

```

<application
    android:allowBackup="false"    // ADD THIS LINE
    android:label="@string/app_name"
    ... >

```

Issue: android:allowBackup=true — anyone with USB can steal app data

WARNING

✅ Fix: Set android:allowBackup="false" in AndroidManifest.xml

<application android:allowBackup="false" ...>

Issue #4 **WARNING — A Background Service Is Open to Other Apps**

What this means in simple words:

Your app has a 'service' (a background process) called DelegationService. Because of how it is set up, ANY other app on the phone can send commands to it. This is like leaving a door unlocked in your house — other apps could poke around or misuse it.

How to fix it — in your AndroidManifest.xml, find your DelegationService and add exported=false:

```
<service
  android:name=".DelegationService"
  android:exported="false" >    // ADD THIS LINE
  <intent-filter>
    ...
  </intent-filter>
</service>
```

Issue: DelegationService is accessible to any app on the device

WARNING

✅ Fix: Add android:exported="false" to DelegationService in AndroidManifest.xml

```
<service android:name=".DelegationService" android:exported="false">
```

Issue #5 WARNING — Broadcast Receiver Permission Not Properly Defined

What this means in simple words:

Your app has a component called ProfileInstallReceiver. It claims to be protected by a permission (android.permission.DUMP), but that permission is not properly defined in your app. This creates confusion — another malicious app could figure out how to interact with it.

How to fix it — in AndroidManifest.xml, either remove the exported flag or lock it down:

```
<receiver
  android:name="androidx.profileinstaller.ProfileInstallReceiver"
  android:exported="false" />    // simplest fix: set to false
```

Issue: Broadcast Receiver permission level not properly verified

WARNING

✅ Fix: Set android:exported="false" on ProfileInstallReceiver in AndroidManifest.xml

```
<receiver android:name="...ProfileInstallReceiver" android:exported="false" />
```

Issue #6 WARNING — Insecure Random Number Generator in Code

What this means in simple words:

Somewhere in your app's code (in a file called J/b.java), you are using a function called `Random()` to generate random numbers. The problem is this function is NOT truly random — it is predictable. If your app uses these numbers for anything security-related (like generating tokens, IDs, or session keys), a hacker could guess them.

How to fix it — find all uses of `java.util.Random` in your code and replace them:

```
// ❌ REMOVE THIS (insecure):  
Random random = new Random();  
  
// ✅ USE THIS INSTEAD (secure):  
import java.security.SecureRandom;  
SecureRandom random = new SecureRandom();
```

Issue: `java.util.Random` used — predictable, not safe for security purposes

WARNING

✅ Fix: Replace `java.util.Random` with `java.security.SecureRandom` in J/b.java

```
SecureRandom random = new SecureRandom(); // replace new Random()
```

Issue #7 INFO — App Is Logging Sensitive Information

What this means in simple words:

Your app is writing information to Android's log system (like a diary the phone keeps). On a normal phone this is fine, but on a rooted phone, other apps can read this diary. If sensitive data (like user actions or internal state) ends up in these logs, it could be a privacy risk.

How to fix it — before releasing your app, disable all logging. In Android, the easiest way is to check if you are in debug mode:

```
// ❌ Don't do this in production:  
Log.d("TAG", "User data: " + userData);  
  
// ✅ Do this instead:  
if (BuildConfig.DEBUG) {  
    Log.d("TAG", "User data: " + userData);  
}
```

Issue: Sensitive information written to Android logs

INFO

✅ Fix: Wrap all Log.d / Log.e calls in if (BuildConfig.DEBUG) check

```
if (BuildConfig.DEBUG) { Log.d("TAG", message); }
```

4. What Is Already Good

Your app does several things correctly. These are worth keeping:

- HTTPS is used for all network connections — data travelling between the app and servers is encrypted.
- No hardcoded passwords or API keys were found hidden in the code.
- No malware was detected.
- Firebase database has no exposed or misconfigured endpoints.
- No shared native libraries with dangerous binary weaknesses.

5. Step-by-Step Fix Checklist

Use this as a to-do list. Fix them in order from top (most urgent) to bottom:

#	What to Fix	Priority	Done?
1	Change minSdkVersion 23 to 29 in build.gradle	 HIGH	☐
2	Disable v1 signing, enable v2/v3 in build.gradle	 Medium	☐
3	Set android:allowBackup="false" in AndroidManifest.xml	 Medium	☐
4	Set android:exported="false" on DelegationService	 Medium	☐
5	Set android:exported="false" on ProfileInstallReceiver	 Medium	☐
6	Replace Random() with SecureRandom() in J/b.java	 Medium	☐
7	Wrap all Log.d calls in BuildConfig.DEBUG check	 Low	☐

6. Files to Edit — Quick Reference

If you are not sure which file to open, here is a quick map:

File	What to Change There
<code>app/build.gradle</code>	minSdkVersion and signing config (Issues 1 & 2)
<code>app/src/main/AndroidManifest.xml</code>	allowBackup, exported flags (Issues 3, 4 & 5)
<code>J/b.java</code> (in source code)	Replace Random() with SecureRandom() (Issue 6)
Any file with <code>Log.d</code> or <code>Log.e</code>	Wrap in BuildConfig.DEBUG check (Issue 7)

After making all these changes, rebuild your APK and share it for a re-scan to confirm everything is fixed. All 7 issues above are straightforward code changes — no advanced security knowledge is needed to fix them.

Report prepared using MobSF v4.5.0 • DATA247.apk • June 8, 2026